

AIOps

Guía paso a paso para construir un agente

Orquestador, workers, fiscalizador, embeddings, SQL, GitHub y GCP

Advanced Prompt Engineering

Clase complementaria · Arquitectura multi-agente end-to-end

Roadmap

10 pasos para llevar un agente a producción

UNIVERSIDAD
ADOLFO IBÁÑEZ

01

Documentos
de negocio

02

Embeddings
→ Vector DB

03

Agente
Orquestador

04

Agentes
Workers

05

Agente
Fiscalizador

06

Búsqueda
semántica (KNN)

07

SQL
para registros

08

Subir
a GitHub

09

Conectar
GCP Cloud Run

10

Deploy
& Go Live

¿Qué documentos preparar?

- **Políticas:** reglas, normativas, compliance interno.
- **Procesos:** flujos paso a paso, SLAs, escalamientos.
- **FAQs:** preguntas frecuentes y respuestas oficiales.
- **Catálogo:** productos, servicios, glosario interno.
- **Casos:** ejemplos reales, decisiones previas.
- **Manuales:** guías técnicas y operacionales.

Buenas prácticas de preparación

- **Formato limpio:** Markdown o .txt, sin tablas raras.
- **Chunks lógicos:** una idea por sección, 300–800 tokens.
- **Metadata:** área, versión, fecha, owner, sensibilidad.
- **Sin duplicados:** una sola fuente de verdad por tema.
- **Versionado:** guarda en Git para trazabilidad.
- **Revisión humana:** experto valida antes de indexar.

Regla de oro: garbage in → garbage out. La calidad del corpus define el techo del agente.

Embeddings → Vector DB

Transformar texto en vectores semánticos

UNIVERSIDAD
ADOLFO IBÁÑEZ

PASO 02

Pipeline

- **1. Load:** leer los documentos del corpus.
- **2. Split:** chunking por tokens o por estructura.
- **3. Embed:** modelo (OpenAI, Voyage, Cohere).
- **4. Store:** insertar vector + metadata.
- **5. Index:** indexar para búsqueda ANN/KNN.

```
# Python pseudo-código
chunks = split(doc, size=500, overlap=50)
vectors = embed_model.encode(chunks)
vector_db.upsert(vectors, metadata)
```

Vector DB: opciones



Comparativa rápida

- **Pinecone** — managed, fácil de empezar
- **Weaviate** — open-source, híbrida
- **Qdrant** — rápido, self-host o cloud
- **pgvector** — Postgres + embeddings
- **Chroma** — local, ideal para PoC

Rol y responsabilidades

- **Interpretar:** entender la intención del usuario.
- **Planificar:** descomponer en sub-tareas.
- **Delegar:** enrutar a los workers correctos.
- **Consolidar:** unificar respuestas de workers.
- **Decidir:** cuándo pedir aclaración o cerrar.
- **Loggear:** guardar la traza en SQL.

Prompt template (ejemplo)

ROL: Eres el agente orquestador de AIOps.

OBJETIVO: Resolver la consulta del usuario delegando en workers especializados.

WORKERS DISPONIBLES:

- worker_kb -> consulta base conocimiento
- worker_sql -> consulta datos transaccionales
- worker_action -> ejecuta acciones (tickets, mail)

REGLAS:

1. Analiza la pregunta y descompón si es necesario.
2. Llama a 1+ workers; nunca inventes datos.
3. Si la info es insuficiente -> pide aclaración.
4. Devuelve respuesta final + IDs de trazabilidad.

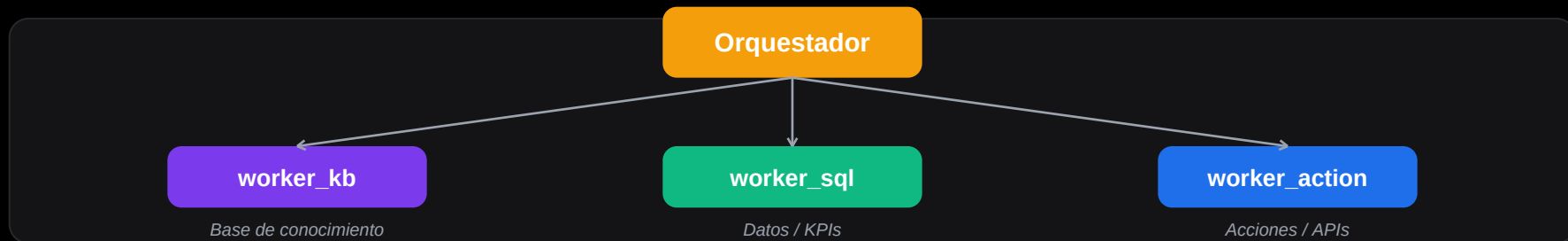
FORMATO DE SALIDA: JSON con {answer, sources, calls}

Agentes Workers

Especialistas que ejecutan tareas concretas

UNIVERSIDAD
ADOLFO IBÁÑEZ

PASO 04



Prompt típico de un worker (ej. worker_kb)

ROL: Worker especialista en base de conocimiento corporativa.

INPUT: { query: str, top_k: int=5, filters: {area, version} }

PROCESO:

1. Embebe la query y haz búsqueda KNN en la vector DB.
2. Recupera top_k chunks con su metadata y score.
3. Construye respuesta CITANDO fuentes (id, archivo, página).

RESTRICCIONES: si confianza < 0.6 -> responde 'sin evidencia suficiente'.

OUTPUT: { answer, citations[], confidence }

PASO 05

¿Qué fiscaliza?

- **Veracidad:** ¿la respuesta está soportada por las fuentes?
- **PII:** detecta datos sensibles que no deben salir.
- **Tono:** respeta el tono de marca y compliance.
- **Alucinaciones:** marca afirmaciones sin evidencia.
- **Política:** bloquea respuestas fuera de scope.
- **Logs:** registra veredicto en SQL para auditoría.

Prompt del fiscalizador

```
ROL: Eres el agente fiscalizador (auditor).

RECIBES: { user_query, draft_answer, citations }

EVALÚA (0 a 1):
- groundedness: ¿está sustentado en citations?
- safety:       ¿contiene PII o contenido riesgoso?
- policy_fit:   ¿respeta políticas de la empresa?
- tone:         ¿tono profesional adecuado?

DECISIÓN:
- APROBAR      -> devolver al usuario
- CORREGIR     -> regresar a orquestador con feedback
- BLOQUEAR     -> respuesta canned de fallback

OUTPUT: { verdict, scores, reasons }
```

Búsqueda semántica (KNN)

La función que conecta el agente con el conocimiento

UNIVERSIDAD
ADOLFO IBÁÑEZ

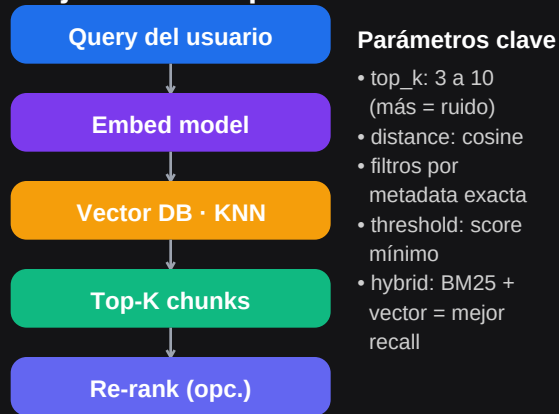
PASO 06

¿Cómo funciona KNN semántico?

- 1. La query se transforma en vector con el mismo embed model.
- 2. Se calcula similitud (coseno) vs. vectores indexados.
- 3. Se devuelven los top-K más cercanos + metadata.
- 4. Opcional: re-ranking con cross-encoder.
- 5. Filtros por metadata (área, versión, fecha).

```
def semantic_search(query, top_k=5, filters=None):  
    qv = embed_model.encode(query)  
    hits = vector_db.search(  
        vector=qv, top_k=top_k,  
        filter=filters, include_metadata=True)  
    return [(h.id, h.score, h.metadata) for h in hits]
```

Flujo de la búsqueda



¿Qué registrar?

- **Sesiones:** user_id, channel, timestamp.
- **Interacciones:** query, respuesta, latencia.
- **Llamadas a workers:** qué worker, input, output.
- **Citaciones:** qué documentos sustentaron la respuesta.
- **Veredicto fiscal:** score, decisión, motivos.
- **Feedback:** thumbs up/down del usuario.
- **Errores:** stack traces, reintentos.

Esquema mínimo sugerido

```
-- PostgreSQL
CREATE TABLE sessions (
  id UUID PRIMARY KEY,
  user_id TEXT, channel TEXT,
  started_at TIMESTAMPTZ);

CREATE TABLE interactions (
  id UUID PRIMARY KEY,
  session_id UUID REFERENCES sessions(id),
  query TEXT, answer TEXT,
  latency_ms INT, tokens_in INT, tokens_out INT,
  created_at TIMESTAMPTZ);

CREATE TABLE audit_log (
  interaction_id UUID,
  verdict TEXT,      -- APPROVED|FIX|BLOCKED
  groundedness REAL, safety REAL,
  reasons JSONB);
```

Estructura recomendada del repo

```
agent-aiops/  
+-- app/  
|   +-- main.py           # FastAPI endpoint  
|   +-- orchestrator.py  
|   +-- workers/  
|       |   +-- kb.py  
|       |   +-- sql.py  
|       |   +-- action.py  
|   +-- auditor.py  
|   +-- search.py         # KNN semantico  
+-- prompts/             # *.md por agente  
+-- data/                # corpus (gitignore)  
+-- infra/  
|   +-- Dockerfile  
|   +-- cloudrun.yaml  
+-- tests/  
+-- requirements.txt  
+-- .github/workflows/ci.yml
```

Flujo Git mínimo

- 1. Crear repo privado en GitHub.
- 2. Configurar .gitignore (secrets, .env, data).
- 3. Branch protection en main.
- 4. Pull requests + code review obligatorio.
- 5. GitHub Actions para tests y lint.
- 6. Secrets en GitHub → no en el código.

```
git init && git add .  
git commit -m 'feat: initial agent skeleton'  
git branch -M main  
git remote add origin git@github.com:org/agent-aiops.git  
git push -u origin main
```

¿Por qué Cloud Run?

- **Serverless:** escala a cero cuando no hay tráfico.
- **Contenedores:** deploy de cualquier imagen Docker.
- **Pago por uso:** facturas por request y CPU activa.
- **Autoescala:** de 0 a N instancias automáticamente.
- **HTTPS nativo:** endpoint seguro listo para usar.
- **Secrets Manager:** integración nativa GCP.
- **Logs / Monitoring:** Cloud Logging out-of-the-box.

Dockerfile + deploy

```
# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ ./app
ENV PORT=8080
CMD exec uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

```
# Build & deploy
gcloud builds submit --tag gcr.io/$PROJECT/agent-aiops
gcloud run deploy agent-aiops \
  --image gcr.io/$PROJECT/agent-aiops \
  --region us-central1 \
  --set-secrets OPENAI_API_KEY=openai-key:latest \
  --allow-unauthenticated
```

Deploy & Go Live

Checklist final antes y después de lanzar

UNIVERSIDAD
ADOLFO IBÁÑEZ

PASO 10

Checklist pre-launch

- **Tests:** unitarios + integración + evals del agente.
- **Eval set:** preguntas doradas con respuestas esperadas.
- **Costos:** estimación por 1000 interacciones.
- **Rate limits:** protección contra abuso.
- **Secrets:** OPENAI_KEY, DB_URL en Secret Manager.
- **Observabilidad:** logs, métricas, alertas.
- **Rollback:** plan de reversión claro.

Post go-live: operar el agente

- **Monitor:** latencia P95, tasa de error, costos.
- **Feedback loop:** thumbs up/down → fine-tune prompts.
- **A/B testing:** compara variantes de prompts.
- **Re-index:** actualiza vector DB cuando cambien docs.
- **Audit reviews:** muestra semanal del fiscalizador.
- **Mejora continua:** iterar prompts y workers.
- **SLO/SLA:** compromisos medibles con negocio.

GO LIVE: medir, escuchar, iterar. El agente nunca está terminado.

Gracias!

AIOps · Guía paso a paso para construir un agente

Documentos → Embeddings → Agentes → Search → SQL → Git → Cloud Run → Live